

Tide retrospective: *Generic even-monomial 1D template (A1)*

Daniel Murfet

18 May 2026

Abstract

Programme-level lift of the 1D quartic / sextic template to arbitrary $k \geq 1$. For the centred even-monomial potential $L_k(x) = x^{2k}/(2k)!$, lands the exact closed forms for the partition function, even moments, and expected values, plus odd-moment vanishing. After A1, the quartic and sextic seabed specialisations are mathematically (though not yet syntactically) five-line corollaries. 310 lines, single session, three trailing `ring` / `field_simp` cleanups needed. Zero `sorry`, zero `axiom`, zero `native_decide`. Integrability witnesses (`kth_integrable_pow`) are deliberately deferred to a follow-up tide — the Gaussian-domination argument needs case analysis on $k = 1$ vs $k \geq 2$ and is substantial enough to warrant its own deliberation.

1 Setting

Today's V2 / Z1 / Z2 trio packaged exact 2D quartic-sextic asymptotics; A1 lifts the underlying 1D template to arbitrary k . N1 (the generic- k J-function tide) had already established the *partition function in raw-integral form*; A1 fills the rest of the template (moments, odd-vanishing, partition in `partitionFunction`-form, expected values), matching the shape of the quartic and sextic files structurally.

The strategic value: after A1 lands, downstream programmes using pure-monomial potentials at arbitrary k (or 2D separable combinations of such potentials) can build on a parametric template rather than ad-hoc quartic / sextic specialisations. The today's quartic-sextic 2D story has effectively prototyped what the template enables.

2 The thing formalised

For $k \geq 1$, $j \in \mathbb{N}$, and $t > 0$:

- *Half-line moment*: $\int_0^\infty x^{2j} e^{-tx^{2k}/(2k)!} dx = \frac{1}{2k} \cdot ((2k)!/t)^{(2j+1)/(2k)} \cdot \Gamma((2j+1)/(2k))$.
- *Full-line moment* (`kth_moment_even`): $\int_{\mathbb{R}} x^{2j} e^{-tx^{2k}/(2k)!} dx = \frac{1}{k} \cdot ((2k)!/t)^{(2j+1)/(2k)} \cdot \Gamma((2j+1)/(2k))$.
- *Odd-vanishing* (`kth_moment_odd`): $\int x^{2j+1} e^{-tx^{2k}/(2k)!} dx = 0$.
- *Partition in `partitionFunction`-form*: $Z_{L_k}(t) = \frac{1}{k} \cdot ((2k)!/t)^{1/(2k)} \cdot \Gamma(1/(2k))$.
- *Expected values* (even, odd): moments divided by the partition; even case is $((2k)!/t)^{j/k} \cdot \Gamma_{\text{num}}/\Gamma_{\text{den}}$, odd case vanishes.

Named definition `kthPotential + kthPotential.apply @[simp]`. File: sibling to the quartic and sextic files in `Laplace/OneD/`.¹

3 Proof strategy

The half-line moment is a direct call to Mathlib’s `integral_rpow_mul_exp_neg_mul_rpow` at $p = 2k$, $q = 2j$, $b = t/(2k)!$, with the standard cast-handling massage to convert Nat-power form to rpow form for the application. The full-line moment follows by `integral_comp_abs` on the even-symmetric integrand; the factor of 2 absorbs into $2/(2k) = 1/k$. Odd-vanishing is via odd reflection symmetry against the even-symmetric measure, identical template to `quartic_moment_odd`. Partition is the $j = 0$ specialisation. Expected values are the moment / partition ratio, with an `rpow_add` step to combine $((2k)!/t)^{(2j+1)/(2k)-1/(2k)} = ((2k)!/t)^{j/k}$.

4 Roadblocks and resolutions

Three trailing tactic mismatches at the bottom of the longer proofs:

1. In the half-line integral proof,² an `hg` sub-lemma used `push_cast; ring`, but `push_cast` alone closed the goal so `ring` errored with “No goals to be solved”. The asymmetric `hgneg` needed `ring` because `push_cast` stopped one step short. Fix: `push_cast` alone for the positive case, `ring` after `push_cast` for the negative.
2. Same pattern in two further proofs:³ `field_simp` closed the goal alone in each. Fix: drop the trailing `ring`.
3. In the `partitionFunction_kthPotential` bridge, an `hexp` helper for the exponent reduction used the form $(2 * 0 + 1 : \mathbb{R}) / \dots$ but the post-`kth_moment_even` goal had Nat-cast `↑0` still visible. The `rw` failed to match. Fix: replace the explicit `hexp + rw` with a single `norm_num`, which normalises `↑0` and simplifies $2 * 0 + 1 = 1$ in one pass.

All four were local mid-proof cleanups, not architectural problems. No GPT mid-proof consult was needed; the deliberation at Step 2 set the scope and naming correctly.

5 What was Mathlib, what was new

From Mathlib. The master lemma `integral_rpow_mul_exp_neg_mul_rpow`, `integral_comp_abs`, `integral_neg_eq_self`, the `Real.rpow` algebra suite, `Real.Gamma_pos_of_pos`, and standard `ring / field_simp / push_cast / norm_num`.

From the seabed. Only N1’s `kth_partition` for the raw integral identity is in the spirit; A1 inherits the same proof template (via Mathlib’s master lemma at $q = 0$) and extends to general $q = 2j$. The quartic and sextic seabed lemmas are not load-bearing for A1 (A1 is generic enough to subsume both, even if the actual refactor of the quartic / sextic files is deferred to a follow-up tide).

¹Specifically `Laplace/OneD/MonomialPotential.lean`; per GPT’s preference for descriptive over parametric naming.

²Theorem `integral_pow_mul_exp_neg_kth_Ioi`.

³The full-line `kth_moment_even` and the expected-value lemma both ended with `field_simp; ring`; in both, `field_simp` closed the goal alone.

New here. The named `kthPotential` definition, the full eight-theorem template at parametric k , and the bridge between raw-integral and `partitionFunction` / `gibbsExpectation` forms.

Deferred. Integrability witnesses (`kth_integrable_pow`, `kth_integrable_pow_pot`). The quartic and sextic versions use a Gaussian-domination argument via square completion of $tx^{2k}/(2k)! - x^2$ for $k = 2, 3$ specifically; a generic- k version needs a uniform constant $C(k, t)$ that splits cleanly into the $k = 1$ (direct Gaussian) and $k \geq 2$ (square-completion with parametric constant) cases. That’s substantial enough to warrant its own tide.

6 Lessons

Proceeding without GPT mid-proof works for mirror-shaped extensions. GPT’s pre-proof consult at Step 2 was load-bearing: it voted Candidate A, named the naming triplet, and explicitly recommended deferring the quartic / sextic refactor to a separate tide. Beyond that, A1’s proof is mechanical mirror of the quartic / sextic templates with `rpow/cast` plumbing parametric in k . The deliberation-once + execute-deterministically pattern scales.

field.simp and push.cast are “finishing” tactics in this regime. Four of the proof bodies in this file ended with what was meant to be `push.cast`; `ring` or `field.simp`; `ring`, but the closing `ring` was redundant in three of those four cases — the prior tactic closed the goal. Worth recording: after explicit `rpow_add` / `rpow_neg` normalisations, `ring` is often unnecessary. If `ring` errors with “No goals to be solved”, drop it.

Defer integrability when it forks the proof. The generic Gaussian-domination argument forces a case split on $k = 1$ vs $k \geq 2$ that the quartic / sextic specialisations don’t need. By deferring the integrability layer, A1 stays as a clean uniform parametric lift; the integrability tide can then specialise as needed without contaminating the headline theorems.

7 Follow-ups

- *Integrability tide.* Land `kth_integrable_pow` and `kth_integrable_pow_pot` via the Gaussian-domination argument, case-split on $k = 1$ vs $k \geq 2$. ~80 lines.
- *Refactor quartic and sextic files as specialisations.* After A1 and the integrability follow-up land, rewrite the quartic and sextic files so that their headline theorems become $k = 2$ and $k = 3$ specialisations of the generic ones. Net deletion ~300 lines. Per GPT’s guidance, this is a separate tide.
- *2D analogues at arbitrary k_1, k_2 .* The 2D quartic-sextic story today proved closed for $(k_1, k_2) = (2, 3)$; with A1 in hand, the same V2/Z1/Z2-style packaging extends to arbitrary (k_1, k_2) separable monomial potentials.

Acknowledgements

GPT-5.5 Pro’s Step-2 consult contributed the vote for Candidate A (full template over skinnier variants), the naming triplet, the `MonomialPotential.lean` file name, and the explicit “defer the quartic / sextic refactor” guidance. Full response preserved alongside the tide log in the primer project’s `gpt_responses_a1/` folder.